Lab 2: Cache Emulator

Conner Liddell, Eli Fisk, Paul Blair

GitHub Link: https://github.com/blairpaul8/Cache-Emulator

Implementation Details

For the Implementation of the Cache Emulator we choose to use C++. The cache.hpp file can be referenced to see the main features of the emulator. It contains a cache class that holds the main data structure that represents the cache. It is esentailly a 2-D array that can be thought of as a matrix of n-sets x m-blocks.

```
Block Datastructure

struct Block {
int tag;
bool valid;
chrono::time_point<chrono::steady_clock> timestamp;
int m;
}

Cache Datastructure

vector<vector<Block>> cache;
```

As shown above the we represented the blocks as a struct that contains the tag, valid bit, timestamp, and replacement value m. The timestamp is used for enforcing the least recently used (LRU) replacement policy. The value m is used as the hot cold value, this is used to enforced the RRIP replacement policy. The cache itself is represented as a vector of vectors. With this we can index the outer vector to access a particular set then index the inner vector to access a particular block.

Verification

In order to verify that our cache emulator was working correctly we created a short hand trace file, and computed the hits and misses manually.

Time	Address	Binary	Tag	Set	Set 0	Set 1	Eviction	Hit/Miss
T1	`0x10`	`0001 0 000`	1	0	[**1**,-]	[-,-]		Miss
T2	`0x20`	`0010 0 000`	2	0	[1,**2**]	[-,-]		Miss

T3	`0x10`	`0001 0 000`	1	0	[**1**,2]	[-,-]		**Hit**
T4	`0x30`	`0011 0 000`	3	0	[1,**3**]	[-,-]	Tag 2 (T2)	Miss
T5	`0x20`	`0010 0 000`	2	0	[**2**,3]	[-,-]	Tag 1 (T3)	Miss
T6	`0x30`	`0011 0 000`	3	0	[2,**3**]	[-,-]		**Hit**
T7	`0x48`	`0100 1 000`	4	1	[2,3]	[**4**,-]		Miss
T8	`0x30`	`0011 0 000`	3	0	[2,**3**]	[4,-]		**Hit**
T9	`0x18`	`0001 1 000`	1	1	[2,3]	[4,**1**]		Miss
T10	`0x18`	`0001 1 000`	1	1	[2,3]	[4,**1**]		**Hit**

As seen in the table the hand trace resulted in four hits and six misses with a total of ten memory accesses. The miss rate can be calculated by.

$$MissRate = \frac{\#Misses}{\#MemoryAccesses} = \frac{6}{10} = .60 = 60\% \text{ miss rate}$$

We can compare the results of our cache emulator to verify it is working properly. This can be done by running the following command.

```
.\cache_emulator --blocks 2 --sets 2  --size 8 --trace ./trace_files/verification.trace
```

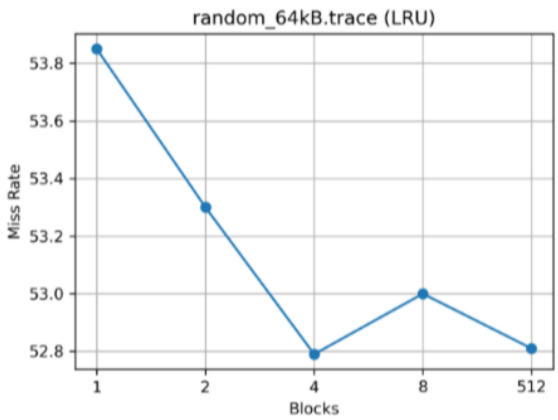
```
Cache-Emulator lab_report ● ? × ./cache_emulator --blocks 2 --sets 2  --size 8 --trace ./trace_files/verification.trace
num_sets: 2
num_blocks: 2
block_size: 8
filename: ./trace_files/verification.trace
RRIP: false

accesses: 10
num_hits: 4
num_misses: 6
Miss Rate: 60.00%
```

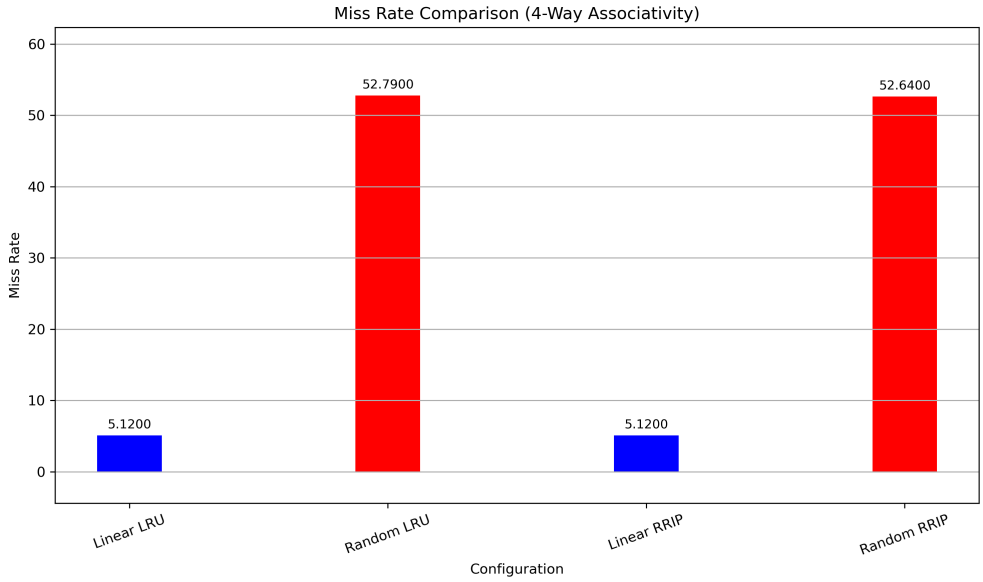
We can see with the output above we get the expected miss rate of 60%

Results

Experiment A: Associativity Sensitivity



Experiment B: The Comparison



Analysis

Our extension (RRIP) performed slightly better on the random trace because it was able to evict the less used values rather than just taking the last used out. They performed the same on the linear trace because the least recently used always matched the RRPV value.

Conclusion

This lab demonstrates how cache performance is strongly influenced by temporal and spatial locality, as seen through comparisons between linear traces (which exhibit high locality and low miss rates) and random traces (which show low locality and high miss rates). Varying the range of memory addresses in random traces (such as 1MB, 64KB, and 32KB) highlights how the relationship between working set size and cache size affects miss rates and the effectiveness of different associativity levels. Increasing associativity generally reduces conflict misses for larger working sets, but can introduce tradeoffs when it reduces the number of sets, sometimes leading to diminished performance in borderline cases. Additionally, comparisons between LRU and RRIP replacement policies show that LRU performs well under strong locality, while RRIP can better manage certain access patterns by predicting reuse, though both behave similarly under highly random workloads. Overall, the results emphasize the importance of workload characteristics and architectural tradeoffs in cache design.